Assignment 2. Advanced DBs

(Kostiantyn Charukovskyi)

Identified issues

	query	calls	total_exec_time	mean_exec_time	rows
1	UPDATE customers SET pho…	12228	31041819.100352906	2538.585140689632	12228
2	UPDATE customers …	2189	4818897.87291	2201.4152000502545	2189
3	CREATE MATERIALIZED VIEW IF NOT …	1	2173.255022	2173.255022	10761
4	UPDATE customers …	6072	11333028.636337033	1866.440816261037	6072
5	UPDATE customers …	9673	7936963.642984984	820.5276173870577	9673
6	SELECT customer_…	28857	3426926.595953998	118.75546993637502	5771400
7	SELECT p.categor…	28952	2659597.2927519903	91.86229941807102	144760
8	SELECT COUNT(*) FROM…	28876	2337033.2440940067	80.93341335690562	28876
9	SELECT c.custome…	28426	1203844.5106509982	42.35011998350127	2842600
10	SELECT * FROM orders…	29051	1057281.6278400053	36.39398395373648	651595645
11	SELECT * FROM custom…	29228	116793.77288699996	3.9959550050294124	0
12	UPDATE customer_events_wide …	31399	12585.204913000056	0.40081546905952503	314457
13	UPDATE orders SET status…	23431	5157.271434999967	0.2201046235756057	23431
14	INSERT INTO customer_events_wide…	200000	4120.451891999989	0.020602259459999803	200000
15	INSERT INTO order_items …	359686	2880.6194049999276	0.008008705940737113	359686

- Issue 1: Lock contention on hot rows. 5 `customers` rows (id 1–5) hit by overlapping transactions; some hold idle-in-transaction for 8–15s.
- Issue 2: Oversized result sets. Analytical query with `LIKE '%a%'` + no LIMIT returns thousands of rows per call.
- Issue 3: Repeated expensive aggregation. `events_aggregation` (GROUP BY on `customer_events_wide`) recomputed ~28k times. Seq Scan + HashAggregate dominate cost; ~49% filter selectivity limits how much indexing/partitioning alone can help.
- Issue 4: Wide table (`customer_events_wide`). 25 columns mixing identity, marketing and device data with 10 attribute fields. Updates to 4 of 25 columns rewrite the whole row.

How detected

- Issue 1: `pg_stat_activity` (idle in transaction / Lock waits) + `pg_stat_statements` (inflated mean times) + deadlock messages in script output.
- Issue 2: `pg_stat_statements`, `rows` column.
- Issue 3: `pg_stat_statements` (call count + total cost) and `EXPLAIN (ANALYZE, BUFFERS)` to break down Seq Scan vs HashAggregate cost.
- VACUUM/ANALYZE check on `customers`: `pg_stat_user_tables` (`n_tup_hot_upd`, `n_dead_tup`, `autovacuum_count`).
- Issue 4: schema inspection (column grouping by purpose) + `EXPLAIN (ANALYZE, BUFFERS)` comparison + `pg_stat_user_tables`

Problematic queries

Issue 1

Query	Calls	Mean	Total
`UPDATE customers SET phone=... WHERE customer_id=$2`	12,007	2,546 ms	~30,571 s
`UPDATE customers SET country=country WHERE customer_id=$1`	5,970	1,876 ms	~11,202 s
`UPDATE customers SET city=city WHERE customer_id=$1`	9,515	818 ms	~7,780 s
`UPDATE customers SET status=$1 WHERE customer_id=$2`	2,151	2,223 ms	~4,781 s

Issue 2

Query	Calls	Mean	Rows (total)
`SELECT * FROM orders WHERE delivery_city LIKE '%a%' AND status=$2`	28,602	36 ms	639,013,208

Issue 3:

Query	Calls	Mean	Total
different GROUP BYs on `customer_events_wide`	28,382	119 ms	~3,379 s

Changes Implemented

- Issue 1a (idle-in-transaction): `ALTER DATABASE student_perf_lab SET idle_in_transaction_session_timeout = '5s';` **No idle transactions longer than 5s.**
- Issue 1b (deadlocks): `deadlock_worker` now sorts `(first_customer_id, second_customer_id)` ascending before locking. Both rows always acquired in the same order, removing the cycle waiting. (Bad application)
- Issue 2 (oversized result): `orders_by_city_and_status` query: added `LIMIT 100`, replaced `SELECT *` with explicit columns. (Bad application)
- Issue 3, attempt 1 (failed): `CREATE INDEX idx_events_event_time ON customer_events_wide(event_time);` — planner did not use it. Cause: `event_time` is generated as a random timestamp, so the matching ~49% of rows are scattered across all pages, basically, nothing for an index to skip.

- Issue 3, attempt 2 (partial): converted to RANGE partitioning by `event_time` (quarterly). Partition pruning confirmed (`Subplans Removed: 2`), but gain limited: the HashAggregate cost is unaffected by partitioning.
- Issue 3, attempt 3 (adopted): materialized view `mv_events_aggregation` + unique index on `(customer_id, event_type)` (required for `CONCURRENTLY` , as Claude recommended to me), refreshed every 10 min via `pg_cron` (`cron.schedule_in_database` , since `cron.database_name` is server-wide and already pinned to bank system DB). Removes the recurring aggregation cost for reads entirely.
- VACUUM/ANALYZE investigation (ruled out): checked `customers` for fillfactor/HOT-update tuning potential, given heavy concentrated writes on 5 hot rows. `pg_stat_user_tables` : HOT ratio 57,176/58,430 ≈ 97.85%, dead tuples 1,573/20,000 (~7.9%), `autovacuum_count=1` . Already optimal.
- Issue 4 (normalization): split `customer_events_wide` into `events (id/customer_id/type/time),` event*marketing*` *(source/campaign/referrer/utm*)*, `event_device` (device/browser/os/ip/page_url), `event_attributes` (attr_01-10 + customer_id, the part `wide_table_update` touches).

Results

- Issue 2 (oversized result): confirmed. New query text shows 100 rows per call,every time (was ~22,000-24,000 rows/call before). `mean_exec_time` also dropped from 36.87 ms to 0.22 ms.
- Issue 3: indexing alone gave no measurable improvement. Partitioning gave a real but partial improvement. Materialized view eliminates the repeated cost for read.
- Issue 4: latency test inconclusive — `event_attributes` was't faster than `customer_events_wide` for the same update. Real benefit is architectural.

Bonus

Issue 1b (deadlocks): combined-fix run confirmed `pg_stat_database.deadlocks` stayed the same (5,335) across two separate readings taken minutes apart, with zero `DEADLOCK detected` messages in the load generator's output over the same window.

Before / after

Issue 1

Query	Before	After (new calls only)
`UPDATE customers SET phone=...`	2546 ms	331 ms
`UPDATE customers SET country=country`	1876 ms	946 ms
`UPDATE customers SET city=city`	818 ms	328 ms

Issue 2:

Variant	Rows/call	Mean exec time
Original (`SELECT *` , no `LIMIT`)	~22,000-24,000	36.87 ms
Fixed (explicit columns + `LIMIT 100`)	100	0.22 ms

Issue 3:

Variant	Execution Time	Buffers
Original (Seq Scan, no index)	153.6 ms	9725
+ index on `event_time`	~same	9725
+ quarterly partitioning	106.6 ms	6117
+ materialized view	6 ms per read	-

AI Assistance Disclosure

This work was completed with assistance from Claude , used as an interactive tutor throughout the diagnostic and optimization process.

- Claude refreshed the PostgreSQL concepts underlying this analysis
- Claude authored the SQL/Python fixes implemented in this report`
- I ran every diagnostic query and test against the live database myself, captured all evidence (snapshots, execution plans, before/after numbers), made analytics and implementation decisions and wrote a draft for this report with further polishing by Claude.